

PROBLEM STATEMENT

# Apple HBR Report Document Q&A

## Agentic AI Foundations

October 4, 2026

Sarah Wolberg

# Contents / Agenda (1 of 3)

## 1. Executive Summary and Business Context Slides 5–20

- Executive summary: slides 5–8
- Introduction, problem statement, questions and data: slides 9–11
- Methodology in five steps and solution approach: slides 12–20

## 2. Question Answering using LLM (baseline) Slides 21–37

- Purpose, model and code: slides 21–22
- Questions 1–3: answer, output, observations: slides 23–31
- Evaluation, observations and summary: slides 32–37

## 3. Question Answering with Prompt Engineering Slides 38–58

- Purpose, system prompt, function, settings, parameter choices: slides 38–42
- Questions 1–3: answer, output, observations: slides 43–51
- Evaluation, observations and summary: slides 52–58

# Contents / Agenda (2 of 3)

## 4. Data Preparation for RAG Slides 59–66

- Purpose and dataset: slides 59–60
- Splitting, embeddings, vector database, retriever: slides 61–63
- Observations and summary: slides 64–66

## 5. Question Answering using RAG Slides 67–83

- Purpose and prompts: slides 67–68
- Questions 1–3: answer, output, observations: slides 69–77
- Evaluation, observations and summary: slides 78–83

## 6. Comparison and Business Insights Slides 84–92

- Comparison of the three approaches: slides 84–87
- Observations, business impact, why it matters: slides 88–90
- Business insights and other industries: slides 91–92

# Contents / Agenda (3 of 3)

## 7. Recommendations, Future Study and Conclusion Slides 93–97

- Overall recommendations: slide 93
- Possible applications and future study: slide 94
- Single-pass RAG vs agentic RAG: slide 95
- Metadata and data freshness: slide 96
- Conclusion: slide 97

## 8. Appendix Slides 98–107

- Data background: slide 99; project instructions and steps: slide 100
- Code for baseline, prompting, data preparation and RAG: slides 101–107

# Executive Summary (1 of 4)

Analysts lose time searching long reports for answers. We tested three ways of questioning the 11-page HBR article “How Apple Is Organized for Innovation” with gpt-4o-mini: a plain LLM, an LLM with a system prompt, and retrieval-augmented generation (RAG). The plain LLM could not name the authors and the prompt-engineered LLM named the wrong ones. RAG retrieved the top four chunks of the article first, correctly named Joel M. Podolny and Morten T. Hansen, and cited the supporting pages.

**Actionable insights.** Prompt engineering improved the format of the answers but not their facts, so grounding in the source document is what makes answers reliable. Page citations let analysts verify every claim, and answer quality is limited by retrieval quality.

**Recommendations.** Deploy RAG for analysts who review long reports, keep page citations, and keep a human check on high-impact decisions. Tune chunk size, k and reranking, add metadata, keep source PDFs current and test agentic RAG before extending to multiple documents.

## Executive Summary (2 of 4)

**Question 1: Authors and publisher.** The baseline and prompt-engineered models could only answer from prior knowledge. RAG retrieves the article's title and publication information directly from the source PDF, which makes the answer verifiable rather than merely plausible.

**Question 2: Three leadership characteristics.** This question shows particularly strong retrieval performance. The vector search retrieved the passage that explicitly lists the three characteristics, which shows that semantic retrieval can find a precise answer inside a larger business report.

**Question 3: Leadership and successful innovation.** This is a more complex question because the answer may need several passages. RAG can retrieve Apple's functional structure, expert-led decision rights, attention to product detail, and the dual-lens camera and portrait-mode decision. The LLM then synthesizes them into a concise explanation grounded in the report.

## Executive Summary (3 of 4)

**Overall RAG assessment.** RAG is the strongest of the three tested approaches because it combines semantic retrieval, prompt engineering, source-document context, deterministic generation, metadata preservation and natural-language response generation.

The resulting architecture is better suited to business-document analysis because the model is no longer expected to recall every relevant fact from training. Instead, the system retrieves relevant evidence when it is needed and uses the LLM to interpret, summarize and communicate that evidence.

**This directly supports the project objective:** reducing the time analysts spend manually searching long reports while improving the reliability and usefulness of the insights.

## Executive Summary (4 of 4)

**Important limitations.** RAG improves factual grounding, but it does not eliminate all risk of error. The quality of a RAG response depends heavily on the quality of retrieval: if the correct passage is not retrieved, the model cannot reliably use information that is absent from its context.

The similarity search also returned duplicated or highly overlapping chunks. Overlap helps preserve context, but duplicate retrieval uses context-window space without adding new information. A production version could improve this by deduplicating retrieved chunks, testing different values of  $k$ , using Maximum Marginal Relevance (MMR), adding metadata filters, or reranking passages before they reach the LLM.

**RAG should therefore be viewed as a major improvement in evidence-based generation, but not as an automatic guarantee of correctness.**

# Introduction: Business Context

## The challenge

- Analysts must work through large volumes of reports, research papers and strategic documents
- A dense report, such as an HBR article, takes significant effort to review and details get missed

## The opportunity

- Semantic search and RAG let analysts ask natural-language questions and get answers from the source report
- Less manual effort leaves more time for strategic thinking and faster, better decisions

**Example:** an analyst at a firm like Andreessen Horowitz asks, “How does Apple structure its teams for innovation?” and RAG answers from the source instantly.

**Why it matters:** less manual searching means faster, better-informed decisions.

# Problem Statement and Purpose

## Problem statement

Business analysts must find the right insights in long, information-dense reports. Manual review takes significant effort, slows the analysis and increases the chance of missing details that influence key business decisions.

RAG addresses this by retrieving only the passages that answer each question.

## Purpose

Build a RAG prototype that lets analysts extract insights from long reports without reading them in full. It will:

- Surface only the most relevant content
- Answer queries and summarize key insights
- Support natural-language questions
- Show gains in speed, accuracy and productivity

**Why it matters:** analysts spend less time searching and more time on strategy.

# Questions to Answer and Data

## Common questions to answer

1. Who are the authors of this article and who published this article?
2. List down the three leadership characteristics in bulleted points and explain each one of the characteristics under two lines.
3. Can you explain specific examples from the article where Apple's approach to leadership has led to successful innovations?

**Data:** “How Apple Is Organized for Innovation”, a Harvard Business Review article (November–December 2020) provided as an 11-page PDF. It covers Apple’s organizational design, leadership principles and decision-making processes.

**Why it matters:** the same three questions test each approach against the source article.

# Methodology: Overview

| Step                        | Key settings                          | Why we move to the next step                           |
|-----------------------------|---------------------------------------|--------------------------------------------------------|
| <b>1 Baseline LLM</b>       | gpt-4o-mini, temperature 0.7          | It cannot see the article, so test better instructions |
| <b>2 Prompt engineering</b> | System prompt, temperature 0.2        | Format improved, facts did not, so add the article     |
| <b>3 Prepare the data</b>   | 700-token chunks, 100 overlap, Chroma | The index is built, so retrieve from it                |
| <b>4 RAG</b>                | k = 4, temperature 0                  | Run all three on the same questions and compare        |
| <b>5 Compare</b>            | Same three questions throughout       | Decide what to deploy and what to tune                 |

**Why it matters:** one change per step shows what RAG adds.

# Methodology: Step 1, Baseline LLM

| Step                  | What we did                                              | Why this step                                        | Business impact                               |
|-----------------------|----------------------------------------------------------|------------------------------------------------------|-----------------------------------------------|
| <b>1 Baseline LLM</b> | Sent the three questions to gpt-4o-mini with no document | Sets a benchmark for what the model knows on its own | Shows the risk: no source, and Q1 was refused |

**Step detail.** We sent the three project questions straight to gpt-4o-mini with no system prompt and no document, using temperature 0.7, max\_tokens 500 and top\_p 1.0. This shows what the model can do from training alone. It would not name the authors, listed generic leadership traits such as vision and empathy, and gave well-known Apple examples that were not taken from the article, so none of the answers could be traced to a source.

**Why we move on:** the model cannot see the article, so test better instructions.

# Methodology: Step 2, Prompt Engineering

| Step                        | What we did                                          | Why this step                                         | Business impact                                 |
|-----------------------------|------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------|
| <b>2 Prompt engineering</b> | Added a system prompt and lowered temperature to 0.2 | Tests whether clearer instructions alone fix the gaps | Format improved, but it named the wrong authors |

**Step detail.** We kept the same model and added a system prompt that set its role as a careful business research assistant, told it not to invent facts, to state uncertainty, and to follow the requested format exactly, with three bullets for the leadership question. Temperature was lowered from 0.7 to 0.2 for more focused answers. The format improved, but the model named the wrong authors and still had no access to the article.

**Why we move on:** better instructions changed the format, not the facts.

# Methodology: Step 3, Prepare the Data

| Step                      | What we did                                | Why this step                            | Business impact                              |
|---------------------------|--------------------------------------------|------------------------------------------|----------------------------------------------|
| <b>3 Prepare the data</b> | Loaded, split, embedded and stored the PDF | Turns the report into a searchable index | Search by meaning, and the index is reusable |

**Step detail.** We loaded the 11-page HBR PDF with PyMuPDFLoader and split it into 18 chunks using a token-based splitter set to 700 tokens with 100 tokens of overlap, so ideas that cross a chunk boundary are not lost. Each chunk was converted into a 1,536-dimension vector with OpenAI embeddings and saved in a Chroma vector database, with its page number kept as metadata.

**Why we move on:** the index is built, so we can retrieve from it.

# Methodology: Step 4, RAG

| Step  | What we did                                       | Why this step                      | Business impact                    |
|-------|---------------------------------------------------|------------------------------------|------------------------------------|
| 4 RAG | Retrieved the top 4 chunks and answered from them | Grounds every answer in the source | Correct authors, checkable by page |

**Step detail.** For each question the retriever ran a similarity search and returned the four closest chunks. Each chunk was labeled with its PDF page and passed to gpt-4o-mini with a RAG system prompt that allows answers only from the retrieved context and asks for page citations. Temperature was set to 0, max\_tokens to 500 and top\_p to 0.95, so answers are consistent and can be verified against the source.

**Why we move on:** all three approaches now run on the same questions.

# Methodology: Step 5, Compare

| Step             | What we did                          | Why this step                           | Business impact                             |
|------------------|--------------------------------------|-----------------------------------------|---------------------------------------------|
| 5 <b>Compare</b> | Judged all nine answers side by side | Shows which approach analysts can trust | Deploy RAG with citations and a human check |

**Step detail.** We compared the baseline, prompt-engineered and RAG answers on accuracy, relevance, format and traceability. Only RAG named the correct authors, listed the article's three leadership characteristics and cited its pages. Prompting changes how the model answers, while RAG changes the evidence it can use. We also noted limits, such as duplicate retrieved chunks, to fix before scaling.

**Result:** one change per step shows what each technique adds to answer quality.

# Business Problem Overview and Solution Approach

## The problem

Analysts must find insights in long reports, such as the 11-page HBR article on Apple. Manual review is slow and details get missed.

## Why it matters

Missed details can affect key decisions.

**Business impact:** faster review, answers checked by page, more time for strategy.

## Solution approach: four steps, one change each

### 1 Baseline

gpt-4o-mini sees only the question

### 2 Prompting

System prompt, temperature 0.2

### 3 Data prep

Chunk, embed and store in Chroma

### 4 RAG

Top 4 chunks, answer with page cites

# Business Problem: Description

**The problem.** Analysts work through large volumes of reports and strategic documents. Finding the right insight in a dense report, such as the 11-page HBR article on Apple, takes significant effort and risks missing details.

**Why it matters.** These insights feed key business decisions, for example at a venture capital firm such as Andreessen Horowitz. Slow or incomplete review delays decisions and lowers their quality.

**Business impact.** Analysts ask questions in plain language and get answers from the source report, so they skip the full read-through, cut manual effort and spend more time on strategy.

# Solution Approach: Description

**What we built.** A retrieval-augmented generation (RAG) prototype that finds the passages of a report most relevant to a question and has gpt-4o-mini answer from them, with page references.

**How we tested.** We asked the same three questions three ways: the model alone, the model with a system prompt, and RAG. Each step added one change, so we could see what it contributed.

**What we found.** Prompting improved format but not facts, and the model named the wrong authors. Only RAG used the article, named Podolny and Hansen, and cited pages.

# Question Answering using LLM: Purpose

**Purpose.** Establish a baseline. We ask the three project questions to gpt-4o-mini with no article and no system prompt, to see what a general-purpose LLM can do from its training alone.

**Why this is important.** Without a baseline we cannot show what prompt engineering and RAG add. It also tests the assumption that an LLM can answer questions about a specific report on its own.

**Business impact.** If analysts trust a plain LLM for report-specific facts, they risk using unsupported or wrong answers in decisions. The baseline measures that risk before we build anything.

# Question Answering using LLM: Model and Code

```
client = OpenAI() # credentials from config.json; test: "API connection successful"
def response(user_prompt, max_tokens=500, temperature=0.7, top_p=1.0):
    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": user_prompt}],
        max_tokens=max_tokens, temperature=temperature, top_p=top_p)
    return completion.choices[0].message.content
```

| Model              | temperature                           | max_tokens            | top_p                               |
|--------------------|---------------------------------------|-----------------------|-------------------------------------|
| <b>gpt-4o-mini</b> | <b>0.7</b>                            | <b>500</b>            | <b>1.0</b>                          |
| OpenAI chat model  | Randomness; 0.7 allows varied wording | Caps the reply length | 1.0 uses the full probability range |

**Prompt:** the user's question only, with no system prompt and no document.

# Question Answering using LLM: Q1 Answer

**Question 1 (exact wording):** Who are the authors of this article and who published this article?

```
question_1 = "Who are the authors of this article and who published this article ?"  
base_prompt_response_1 = response(question_1)
```

**Answer.** "I'm sorry, but I cannot access external content or specific articles, including their authors and publishers. However, if you provide me with the title or a brief description of the article, I might be able to help you with general information or context related to the topic."

**For comparison.** The article names Joel M. Podolny and Morten T. Hansen (Apple University) as authors, and Harvard Business Review as publisher.

# Question Answering using LLM: Q1 Output

Question 1: Who are the authors of this article and who published this article?

```
question_1 = "Who are the authors of this article and who published this article ?"  
base_prompt_response_1=response(question_1)
```

```
base_prompt_response_1
```

```
"I'm sorry, but I cannot access external content or specific articles, including their authors and publishers. However, if you provide me with the title or a brief description of the article, I might be able to help you with general information or context related to the topic."
```

**Notebook cells In [17] and In [18].** The question, the call to response(), and the raw output: the model says it cannot access the article, so it gives no authors or publisher.

# Question Answering using LLM: Q1 Observations

**Observations.** The model refused: it said it cannot access external content and asked for the title or a description. It did not guess, which is honest, but it gave the analyst nothing to use.

**Why it matters.** The authors and publisher are the first facts an analyst needs to cite a source. A model that cannot see the document cannot supply them, so a plain LLM is unsuitable for report-specific questions.

**Business impact.** The analyst must still open the PDF and look the authors up by hand, so no time is saved. It shows why the document itself must be given to the model.

# Question Answering using LLM: Q2 Answer

**Question 2 (exact wording):** List down the three leadership characteristics in bulleted points and explain each one of the characteristics under two lines.

```
question_2 = "List down the three leadership characteristics in bulleted ..."  
base_prompt_response_2 = response(question_2)
```

**Vision:** Effective leaders possess a clear vision that inspires and guides their team toward common goals. This characteristic allows them to set a direction and motivate others to work collaboratively towards achieving it.

**Empathy:** Strong leaders demonstrate empathy by understanding and valuing the emotions and perspectives of their team members. This fosters trust and strengthens relationships, leading to a more cohesive and motivated team.

**Decisiveness:** Good leaders make timely and informed decisions, even in the face of uncertainty. This trait instills confidence among team members and ensures that progress continues, minimizing delays and indecision.

# Question Answering using LLM: Q2 Output

Question 2: List down the three leadership characteristics in bulleted points and explain each one of the characteristics under two lines.

```
In [19]: question_2 = "List down the three leadership characteristics in bulleted points and explain each one of the characteristics under two lines." #Complete the code to define  
base_prompt_response_2=response(question_2) #Complete the code to pass the user input
```

```
In [20]: base_prompt_response_2
```

```
Out[20]: '- **Vision**: Effective leaders possess a clear vision that inspires and guides their team toward common goals. This characteristic allows them to set a direction and motivate others to work collaboratively towards achieving it.\n\n- **Empathy**: Strong leaders demonstrate empathy by understanding and valuing the emotions and perspectives of their team members. This fosters trust and strengthens relationships, leading to a more cohesive and motivated team.\n\n- **Decisiveness**: Good leaders make timely and informed decisions, even in the face of uncertainty. This trait instills confidence among team members and ensures that progress continues, minimizing delays and indecision.'
```

**Notebook cells In [19] and In [20].** The question, the call to response(), and the raw output: three bullets, Vision, Empathy and Decisiveness, each with a short explanation. The \n characters are line breaks in the stored text.

# Question Answering using LLM: Q2 Observations

**Observations.** The format was followed: three bullets, two sentences each. But vision, empathy and decisiveness are generic leadership traits. The article names deep expertise, immersion in the details and willingness to collaboratively debate, so none of the three match.

**Why it matters.** The answer is fluent and well formatted, which makes the error easy to miss. A confident wrong answer is more dangerous than a refusal because nothing signals that it needs checking.

**Business impact.** An analyst who used these traits in a note on Apple would misreport the article's central finding. That harms credibility and can lead to poor decisions.

# Question Answering using LLM: Q3 Answer

**Question 3 (exact wording):** Can you explain specific examples from the article where Apple's approach to leadership has led to successful innovations?

```
question_3 = "Can you explain specific examples from the article where ..."  
base_prompt_response_3 = response(question_3)
```

**Answer (condensed).** It opens “While I don’t have access to a specific article, I can provide a general overview...” and lists five examples:

- **iPod and iTunes:** Jobs’ mix of hardware and software changed digital music.
- **iPhone:** multi-touch interface and the App Store set a new standard.
- **iPad:** a new product category between smartphone and laptop.
- **Apple Watch:** a focus on health and fitness tracking.
- **Sustainability:** a carbon-neutral supply chain goal by 2030.

# Question Answering using LLM: Q3 Output

Question 3: Can you explain specific examples from the article where Apple's approach to leadership has led to successful innovations?

```
In [21]: question_3 = "Can you explain specific examples from the article where Apple's approach to leadership has led to successful innovations?" #Complete the code to define the  
base_prompt_response_3=response(question_3) #Complete the code to pass the user input
```

```
In [22]: base_prompt_response_3
```

```
Out[22]: "While I don't have access to a specific article, I can provide a general overview of how Apple's approach to leadership has historically led to successful innovations. Here are a few notable examples:\n\n1. **iPod and iTunes**: Under Steve Jobs' leadership, Apple revolutionized the music industry with the introduction of the iPod in 2001 and iTunes in 2003. Jobs' vision for a seamless integration of hardware and software allowed users to easily purchase, manage, and enjoy music. This innovation not only transformed the way people listened to music but also established Apple as a leader in digital content sales.\n\n2. **iPhone**: The launch of the iPhone in 2007 is perhaps one of the most significant innovations in technology. Jobs focused on creating a user-friendly interface with a multi-touch screen, which redefined smartphones. Apple's commitment to design and functionality, combined with a robust ecosystem of apps through the App Store, set a new standard in the mobile industry.\n\n3. **iPad**: The introduction of the iPad in 2010 showcased Apple's ability to identify and create new product categories. The leadership team, inspired by the success of the iPhone, envisioned a device that would bridge the gap between a smartphone and a laptop. The iPad has since become a staple in both consumer and professional markets, demonstrating Apple's prowess in anticipating user needs.\n\n4. **Apple Watch**: With the Apple Watch, launched in 2015, Apple ventured into the wearables market. The company's leadership emphasized health and fitness tracking, which resonated with consumers. The watch's integration with iOS and the focus on user experience led to it becoming the best-selling smartwatch.\n\n5. **Sustainability Initiatives**: More recently, Apple has taken a leadership role in sustainability, aiming for a carbon-neutral supply chain by 2030. This commitment not only aligns with global environmental goals but also appeals to consumer values. Innovations in recycling and renewable energy demonstrate how Apple's leadership is focused on long-term impact rather than short-term gains.\n\nIn each of these cases, Apple's leadership has emphasized a strong vision, an understanding of consumer needs, and a commitment to design and usability, which have all contributed to successful innovations."
```

**Notebook cells In [21] and In [22].** The question, the call to response(), and the raw output: it opens by saying it has no access to the article, then lists five general examples. Nothing in it is taken from the HBR article.

# Question Answering using LLM: Q3 Observations

**Observations.** The model said it had no access to the article, then listed well-known Apple products from its training data. It missed the article's own examples: Roger Rosner's software team, the 600-plus camera experts and the iPhone 7 Plus portrait mode.

**Why it matters.** The answer is plausible and well organized but describes Apple in general, not the article's argument that experts lead experts. A reader cannot tell which parts come from the source.

**Business impact.** Generic answers add nothing an analyst could not find publicly. The value of a report review lies in source-specific evidence that can be cited and defended.

# Question Answering using LLM: Evaluation

| Criterion           | Q1 Authors     | Q2 Traits        | Q3 Examples          |
|---------------------|----------------|------------------|----------------------|
| <b>Accuracy</b>     | None given     | Wrong traits     | Not from article     |
| <b>Relevance</b>    | Refused        | Partly relevant  | Partly relevant      |
| <b>Completeness</b> | Incomplete     | Complete in form | Detailed, off source |
| <b>Instructions</b> | Not applicable | Followed format  | Gave a list          |
| <b>Traceability</b> | None           | None             | None                 |

**Business impact:** a plain LLM gives fluent answers with no source, so every claim needs manual checking. That is why we move to prompt engineering, then RAG.

# Observations: Baseline LLM (1 of 4)

**What was done.** The three project questions were sent directly to the OpenAI API using gpt-4o-mini. The model answered from its pretrained knowledge and the wording of each question alone.

## At this stage

- The Apple HBR PDF had not yet been loaded into the model.
- No retrieval process or vector database was used.
- No document chunks were supplied as context.
- No RAG workflow had been implemented.
- The responses therefore represent a baseline LLM-only approach.
- They were evaluated for accuracy, relevance, completeness, formatting and adherence to instructions.

## Observations: Baseline LLM (2 of 4)

**Question 1.** This question required precise factual information about the article's authors and publisher. Although the model generated an answer, the response could not yet be directly verified or traced back to the assigned HBR document.

**Question 2.** This question tested instruction-following by requiring exactly three leadership characteristics in bullet form with brief explanations. It provides a useful baseline for evaluating whether prompt engineering improves formatting, conciseness and compliance with the requested structure.

## Observations: Baseline LLM (3 of 4)

**Question 3.** This question required the model to connect Apple's leadership approach with specific examples of successful innovation. The baseline model generated explanatory content, but because it had not been given the source article, some examples or relationships may reflect its general pretrained knowledge rather than information retrieved from the assigned document.

**Traceability and risk.** Because no source document or retrieved context was provided, the baseline responses have no source traceability and carry a greater risk of unsupported claims or hallucination. The model may produce plausible and coherent answers, but they cannot yet be confirmed as grounded in the HBR article.

## Observations: Baseline LLM (4 of 4)

**Stored for later comparison.** The generated responses were saved in these three variables so they can be compared with the prompt-engineered and RAG answers.

```
base_prompt_response_1, base_prompt_response_2, base_prompt_response_3
```

**Overall.** This baseline establishes a benchmark for the next stages. Prompt engineering will be evaluated to see whether more detailed instructions and parameter tuning improve structure, relevance and instruction-following. RAG will then be evaluated to see whether retrieving information directly from the HBR document improves factual grounding, reliability and source traceability.

# Question Answering using LLM: Summary

**Model:** gpt-4o-mini **max\_tokens:** 500 **temperature:** 0.7 **top\_p:** 1.0

**Prompt:** the question only, no document supplied

| Question                        | Answer                                                     | Observation                                                       |
|---------------------------------|------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Q1 Authors and publisher</b> | Could not answer; said it cannot access external content   | No source access, so no authors or publisher named                |
| <b>Q2 Leadership traits</b>     | Vision, Empathy, Decisiveness                              | Generic leadership traits, not the article's                      |
| <b>Q3 Innovation examples</b>   | iPod and iTunes, iPhone, iPad, Apple Watch, sustainability | Plausible, but from pretrained memory and not tied to the article |

# Prompt Engineering: Purpose

**Purpose.** Test whether a detailed system prompt and a lower temperature improve the model's answers. We keep the same model, gpt-4o-mini, and the same three questions, and change only the instructions and the temperature.

**Why this is important.** Prompting is the cheapest fix: it needs no new data or infrastructure. If it solved the baseline's errors, a plain model would be enough. The test shows what instructions can and cannot fix.

**Business impact.** Teams often try better prompts first. This step shows the limit: format and tone improve, but facts about a specific report still come from memory, so more prompt tuning alone will not make answers verifiable.

# Prompt Engineering: System Prompt

**Model:** gpt-4o-mini **max\_tokens:** 500 **temperature:** 0.2 **top\_p:** 1.0

Change from baseline: system prompt added, temperature lowered from 0.7 to 0.2

## System prompt

- **Role:** careful business research assistant for the HBR article “How Apple Is Organized for Innovation”
- **Answer directly,** do not invent facts, and state any uncertainty
- **Follow the requested format exactly:** three bullets of two lines each for the leadership question
- **Innovation questions:** give specific examples and explain how leadership contributed
- **Prioritize** accuracy, relevance and clarity over creativity

# Prompt Engineering: System Prompt Code

## Question Answering using LLM with Prompt Engineering

In the next step, we will use prompt engineering to check the effect of a more detailed and well-engineered prompt on the output of the model.

```
31: system_prompt = """
You are a careful business research assistant answering questions about the
Harvard Business Review article "How Apple Is Organized for Innovation."

Provide accurate, concise, and well-structured responses.

Follow these instructions:
1. Answer the user's question directly.
2. Do not invent facts or unsupported details.
3. If you are uncertain about a factual detail, clearly state the uncertainty.
4. Follow the requested response format exactly.
5. For the leadership characteristics question, provide exactly three bullet points.
6. Keep each leadership-characteristic explanation concise and within two lines.
7. For questions about innovation, identify specific examples and explain how
   Apple's leadership approach contributed to the innovation.
8. Prioritize factual accuracy, relevance, clarity, and completeness over creativity.
""" #Complete the code to define the system prompt
```

**Notebook cell.** The system prompt sets a role (careful business research assistant), a no-invention rule, an uncertainty rule and format rules, including exactly three bullets for the leadership question and specific examples for innovation questions.

# Prompt Engineering: Function and Settings

## Defining the function to Generate a Response From the LLM

```
In [24]: # Define a function to get a response from the OpenAI chat model
def response(system_prompt, user_prompt, max_tokens=500, temperature=0.2, top_p=1.0): # Complete the code to set default parameters
    # Create a chat completion using the OpenAI client
    completion = client.chat.completions.create(
        model="gpt-4o-mini", # Complete the code by specifying the model to be used.
        messages=[
            {"role": "system", "content": system_prompt}, # System prompt sets the assistant's behavior
            {"role": "user", "content": user_prompt} # User prompt is the input/query to respond to
        ],
        max_tokens=max_tokens, # Max number of tokens to generate in the response
        temperature=temperature, # Controls randomness in output (0 = deterministic)
        top_p=top_p # Controls diversity via nucleus sampling
    )
    return completion.choices[0].message.content # Return the text content from the model's reply
```

| Setting                  | Baseline              | Prompt engineering         |
|--------------------------|-----------------------|----------------------------|
| Temperature              | 0.7                   | 0.2 (more focused)         |
| System prompt            | None                  | Careful research assistant |
| Model, max_tokens, top_p | gpt-4o-mini, 500, 1.0 | Unchanged                  |

# Prompt Engineering: Parameter Choices

| Setting              | Baseline    | Prompting   | Why                                                       |
|----------------------|-------------|-------------|-----------------------------------------------------------|
| <b>temperature</b>   | 0.7         | 0.2         | Changed: a factual task needs focused, repeatable answers |
| <b>max_tokens</b>    | 500         | 500         | Unchanged, so length limits do not explain differences    |
| <b>top_p</b>         | 1.0         | 1.0         | Unchanged: temperature already controls randomness        |
| <b>model</b>         | gpt-4o-mini | gpt-4o-mini | Unchanged, so only the prompt and temperature differ      |
| <b>system prompt</b> | None        | Added       | Sets role, accuracy rules and the required format         |

**Result:** format improved, facts did not: settings cannot supply the missing article.

# Prompt Engineering: Q1 Answer

**Question 1 (exact wording):** Who are the authors of this article and who published this article ?

```
response_with_prompt_eng_1 = response(system_prompt, question_1)
```

**Answer.** “The article ‘How Apple Is Organized for Innovation’ was authored by Scott D. Anthony, Evan I. Schwartz, and David S. Duncan. It was published by the Harvard Business Review.”

**For comparison.** The article names Joel M. Podolny and Morten T. Hansen (Apple University) as authors. The authors are wrong; the publisher, Harvard Business Review, is correct.

# Prompt Engineering: Q1 Output

Question 1: Who are the authors of this article and who published this article ?

```
In [25]: response_with_prompt_eng_1=response(system_prompt,question_1)
response_with_prompt_eng_1
```

```
Out[25]: 'The article "How Apple Is Organized for Innovation" was authored by Scott D. Anthony, Evan I. Schwartz, and David S. Duncan. It was published by the Harvard Business Review.'
```

```
In [26]: response_with_prompt_eng_1
```

```
Out[26]: 'The article "How Apple Is Organized for Innovation" was authored by Scott D. Anthony, Evan I. Schwartz, and David S. Duncan. It was published by the Harvard Business Review.'
```

**Notebook cells In [25] and In [26].** The call with the system prompt, then the raw output twice: the model names Scott D. Anthony, Evan I. Schwartz and David S. Duncan, so the authors are wrong. Only the publisher is right.

# Prompt Engineering: Q1 Observations

**Observations.** The model answered confidently but named the wrong authors: Scott D. Anthony, Evan I. Schwartz and David S. Duncan. The article lists Joel M. Podolny and Morten T. Hansen of Apple University. The publisher, Harvard Business Review, is correct. The instruction to answer directly removed the baseline's refusal but could not supply the facts.

**Why it matters.** A wrong author in a research note is a credibility error. It is harder to catch than a refusal, because the answer looks complete and carries no source.

**Business impact and action.** Analysts would still check every attribution by hand. Do not rely on prompting alone for citations; give the model the source document.

# Prompt Engineering: Q2 Answer

**Question 2 (exact wording):** List down the three leadership characteristics in bulleted points and explain each one of the characteristics under two lines.

```
response_with_prompt_eng_2 = response(system_prompt, question_2)
```

**Visionary Thinking:** Leaders at Apple possess a clear and compelling vision that drives innovation and aligns the team towards common goals.

**Collaborative Mindset:** Apple leaders foster a culture of collaboration, encouraging cross-functional teamwork to leverage diverse perspectives and expertise.

**Decisive Action:** They demonstrate the ability to make swift and informed decisions, enabling the company to respond quickly to market changes and seize new opportunities.

# Prompt Engineering: Q2 Output

Question 2: List down the three leadership characteristics in bulleted points and explain each one of the characteristics under two lines.

```
In [27]: response_with_prompt_eng_2=response(system_prompt, question_2) #Complete the code to pass the user prompt and system prompt
response_with_prompt_eng_2
```

```
Out[27]: '- **Visionary Thinking**: Leaders at Apple possess a clear and compelling vision that drives innovation and aligns the team towards common goals.\n\n- **Collaborative M
indset**: Apple leaders foster a culture of collaboration, encouraging cross-functional teamwork to leverage diverse perspectives and expertise.\n\n- **Decisive Actio
n**: They demonstrate the ability to make swift and informed decisions, enabling the company to respond quickly to market changes and seize new opportunities.'
```

```
In [28]: response_with_prompt_eng_2
```

```
Out[28]: '- **Visionary Thinking**: Leaders at Apple possess a clear and compelling vision that drives innovation and aligns the team towards common goals.\n\n- **Collaborative M
indset**: Apple leaders foster a culture of collaboration, encouraging cross-functional teamwork to leverage diverse perspectives and expertise.\n\n- **Decisive Actio
n**: They demonstrate the ability to make swift and informed decisions, enabling the company to respond quickly to market changes and seize new opportunities.'
```

**Notebook cells In [27] and In [28].** The call with the system prompt, then the raw output twice: three bullets with one-sentence explanations. The format is exactly as asked, but the traits are generic.

# Prompt Engineering: Q2 Observations

**Observations.** The format was followed exactly: three bullets with one-sentence explanations, shorter than the baseline's. But Visionary Thinking, Collaborative Mindset and Decisive Action are generic. The article names deep expertise, immersion in the details and willingness to collaboratively debate, so none match.

**Why it matters.** Instruction-following improved, which is what prompting can fix. The content is still plausible but wrong, and the clean format makes it look more trustworthy.

**Business impact and action.** Reporting these traits would misstate the article's central finding. Use prompts to control format and tone, and the document to control content.

# Prompt Engineering: Q3 Answer

**Question 3 (exact wording):** Can you explain specific examples from the article where Apple's approach to leadership has led to successful innovations?

```
response_with_prompt_eng_3 = response(system_prompt, question_3)
```

**Answer (shortened; full text on the next slide).** “The article highlights several specific examples where Apple's leadership approach has contributed to successful innovations.”

- 1. iPhone Development:** a culture of collaboration and cross-functional teams enabled the seamless integration of hardware and software.
- 2. Apple Watch:** a focus on user experience and health technology.
- 3. iPad Launch:** encouraging risk-taking and creative thinking supported a new category of devices.

# Prompt Engineering: Q3 Output

Question 3: Can you explain specific examples from the article where Apple's approach to leadership has led to successful innovations?

```
In [29]: response_with_prompt_eng_3=response(system_prompt, question_3) #Complete the code to pass the user prompt and system prompt
response_with_prompt_eng_3
```

```
Out[29]: "The article highlights several specific examples where Apple's leadership approach has contributed to successful innovations:\n\n1. iPhone Development: Apple's leadership fostered a culture of collaboration and cross-functional teams, which enabled the seamless integration of hardware and software, resulting in the revolutionary iPhone.\n\n2. Apple Watch: The leadership emphasized a focus on user experience and health technology, leading to the development of the Apple Watch, which combined fitness tracking with a sleek design, appealing to a broad market.\n\n3. iPad Launch: By encouraging risk-taking and creative thinking, Apple's leadership supported the development of the iPad, which created a new category of devices and transformed how consumers interact with technology.\n\nThese examples illustrate how Apple's leadership approach, characterized by collaboration, user-centric design, and a willingness to innovate, has driven significant product advancements."
```

```
In [30]: response_with_prompt_eng_3
```

```
Out[30]: "The article highlights several specific examples where Apple's leadership approach has contributed to successful innovations:\n\n1. iPhone Development: Apple's leadership fostered a culture of collaboration and cross-functional teams, which enabled the seamless integration of hardware and software, resulting in the revolutionary iPhone.\n\n2. Apple Watch: The leadership emphasized a focus on user experience and health technology, leading to the development of the Apple Watch, which combined fitness tracking with a sleek design, appealing to a broad market.\n\n3. iPad Launch: By encouraging risk-taking and creative thinking, Apple's leadership supported the development of the iPad, which created a new category of devices and transformed how consumers interact with technology.\n\nThese examples illustrate how Apple's leadership approach, characterized by collaboration, user-centric design, and a willingness to innovate, has driven significant product advancements."
```

**Notebook cells In [29] and In [30].** The call with the system prompt, then the raw output twice. It says “the article highlights” three examples, but none comes from the article.

# Prompt Engineering: Q3 Observations

**Observations.** The model now says “the article highlights” the iPhone, Apple Watch and iPad, each with a short link to leadership, as the prompt asked. But none of these examples comes from the article, which the model could not read. It misses Roger Rosner’s software team, the 600-plus camera experts and the iPhone 7 Plus portrait mode.

**Why it matters.** This is the riskiest result: a false claim about the source. The baseline admitted it had no article; the prompted model implies it read one.

**Business impact and action.** A reader could cite examples the article does not contain. Treat any “the article says” claim from a model without retrieval as unverified.

# Prompt Engineering: Evaluation

| Criterion           | Q1 Authors        | Q2 Traits           | Q3 Examples           |
|---------------------|-------------------|---------------------|-----------------------|
| <b>Accuracy</b>     | Wrong authors     | Wrong traits        | Not from article      |
| <b>Relevance</b>    | Answered directly | Partly relevant     | Partly relevant       |
| <b>Completeness</b> | Complete in form  | Complete in form    | Off-source examples   |
| <b>Instructions</b> | Followed          | Followed: 3 bullets | Followed: linked      |
| <b>Traceability</b> | None              | None                | None                  |
| <b>vs baseline</b>  | Refusal → wrong   | Same generic traits | Five → three examples |

**Business impact:** prompting improved format, not accuracy or traceability, so answers still need manual checking. Next: RAG.

# Observations: Prompt Engineering (1 of 5)

**What was done.** The same gpt-4o-mini model was used, but the interaction was improved through a detailed system prompt and a lower generation temperature. Because the underlying model remained unchanged, prompt engineering can be compared fairly with the baseline.

**The system prompt.** It told the model to act as a careful business research assistant and stressed accuracy, relevance, clarity, conciseness and adherence to the requested format. It also added safeguards: avoid unsupported claims and acknowledge uncertainty.

**Compared with the baseline.** The baseline sent each question straight to the model, with no guidance on structure or caution. The prompt-engineered approach added the improvements on the next slide.

## Observations: Prompt Engineering (2 of 5)

**Control over structure.** The system prompt required the requested format, such as exactly three bullets for Question 2.

**Factual accuracy.** The model was told not to invent information and to acknowledge uncertainty.

**Relevance and brevity.** It was told to answer directly and put relevance ahead of creativity.

**Consistent behavior.** The same rules applied to all three questions, giving one role and style.

**Less randomness.** Temperature fell from 0.7 to 0.2, giving more focused, deterministic answers.

**Instruction-following.** Explicit rules covered formatting and linking leadership practices to innovations.

## Observations: Prompt Engineering (3 of 5)

**Question 1.** This question needs precise facts about the authors and publisher. Prompting can encourage a more direct, cautious answer, but the model still relies on pretrained knowledge, not the article. Result: wrong authors.

**Question 2.** This question benefits most, because the output has strict formatting rules. The instruction to give exactly three bullets with short explanations improved compliance. Result: format followed, content generic.

**Question 3.** This question needs factual recall and interpretation. The system prompt makes the model more likely to link leadership to specific innovations. Result: linked, but the examples are not from the article.

# Observations: Prompt Engineering (4 of 5)

**Remaining limitation.** Prompt engineering improves how the model answers, but not where it gets its information. The Apple HBR PDF has still not been supplied, so the model answers from pretrained knowledge.

## As a result

- The answers cannot be verified against the source article.
- There is still no source or page-level traceability.
- Some facts may be incomplete, outdated or from outside the report.
- A plausible but unsupported answer is still possible.

**Key distinction.** Prompting can reduce the risk of poor responses, but it cannot guarantee factual grounding in a specific document.

# Observations: Prompt Engineering (5 of 5)

**Document grounding.** Answers can rest on the actual HBR article, not only pretrained knowledge.

**Less hallucination.** The model is constrained by the retrieved context.

**Document-specific questions.** Facts the model may not know are retrieved from the report.

**Factual reliability.** Retrieved passages give evidence for each response.

**Source traceability.** Retrieved chunks and page metadata show where facts came from.

**Business trust.** Analysts can validate insights against the source before using them.

**Overall.** Prompting improves quality and structure; RAG adds grounding and traceability. The path is a general LLM, a better-controlled LLM, then a source-grounded RAG application.

# Prompt Engineering: Summary

| Question                        | Answer                                                                          | Observation                                                                    |
|---------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Q1 Authors and publisher</b> | Scott D. Anthony, Evan I. Schwartz and David S. Duncan; Harvard Business Review | Wrong authors, the article names Podolny and Hansen; publisher is correct      |
| <b>Q2 Leadership traits</b>     | Visionary thinking, collaborative mindset, decisive action                      | Three bullets as asked, but the traits are generic, not the article's          |
| <b>Q3 Innovation examples</b>   | iPhone development, Apple Watch, iPad launch                                    | Says the article highlights these, but the examples are generic and unverified |

**Prompt engineering improved structure and tone, but the model still answers from memory, not from the article.**

# Data Preparation: Purpose

**Purpose.** Prepare the HBR article for retrieval: load the PDF, split it into chunks, embed each chunk and store the vectors in a database, so the model can later be given only the passages that answer a question.

**Why this is important.** RAG quality depends on this step. Chunk size, overlap and the embedding model decide whether the right passage can be found. Sending a whole report with every question is slow, costly and does not scale to many reports.

**Business impact.** A reusable index lets analysts ask many questions without re-reading the report, and the same pipeline can scale to hundreds of reports. The page numbers kept here are what make answers traceable later.

# Data Preparation: Dataset

**Dataset.** “How Apple Is Organized for Innovation”, Harvard Business Review, November–December 2020 (reprint R2006F). File: HBR\_How\_Apple\_Is\_Organized\_For\_Innovation.pdf, 11 pages.

```
pdf_path = "/content/drive/MyDrive/Apple HBR Report Project/"
           "HBR_How_Apple_Is_Organized_For_Innovation.pdf"
pdf_loader = PyMuPDFLoader(pdf_path)
pdf = pdf_loader.load()    # one document per PDF page
```

**Why this dataset.** It is one dense report. The answers sit on different pages: authors on page 3, the leadership characteristics on page 6 and the innovation examples on pages 7 and 10, so retrieval has to find them inside the text.

# Data Preparation: Splitting Parameters

```
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(  
    encoding_name='cl100k_base', chunk_size=700, chunk_overlap=100)  
document_chunks = pdf_loader.load_and_split(text_splitter)  
len(document_chunks)    # 18
```

| Parameter            | Value       | Role                                 |
|----------------------|-------------|--------------------------------------|
| <b>chunk_size</b>    | 700 tokens  | Maximum size of each chunk           |
| <b>chunk_overlap</b> | 100 tokens  | Tokens shared by neighbouring chunks |
| <b>encoding_name</b> | cl100k_base | Tokenizer used to count tokens       |
| <b>Result</b>        | 18 chunks   | From the 11-page PDF                 |

**Why it matters:** overlap keeps ideas that cross a chunk edge intact.

# Data Preparation: Embeddings and Vector DB

```
embedding_model = OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY,  
                                    openai_api_base=OPENAI_API_BASE)  
embedding_1 = embedding_model.embed_query(document_chunks[0].page_content)  
len(embedding_1)    # 1536  
vectorstore = Chroma.from_documents(document_chunks, embedding_model,  
                                    persist_directory='apple_hbr_chroma_db')
```

| Item                   | Value                           | Why it matters            |
|------------------------|---------------------------------|---------------------------|
| <b>Embedding model</b> | text-embedding-ada-002          | Turns text into vectors   |
| <b>Vector length</b>   | 1,536 per chunk                 | Same space for all chunks |
| <b>Vector database</b> | Chroma<br>(apple_hbr_chroma_db) | Stores vectors and pages  |

# Data Preparation: Retriever and RAG Settings

```
retriever = vectorstore.as_retriever(search_type='similarity',  
                                     search_kwargs={'k': 4})  
def generate_rag_response(user_input, k=4, max_tokens=500,  
                          temperature=0, top_p=0.95):
```

| Parameter          | Value       | Why                                          |
|--------------------|-------------|----------------------------------------------|
| <b>Search type</b> | similarity  | Matches the question to the closest passages |
| <b>k</b>           | 4           | Enough evidence without flooding the prompt  |
| <b>max_tokens</b>  | 500         | Room for a full, cited answer                |
| <b>temperature</b> | 0           | Repeatable answers that stay with the source |
| <b>top_p</b>       | 0.95        | Standard cutoff; temperature 0 does the work |
| <b>Model</b>       | gpt-4o-mini | Same model as the baseline, for a fair test  |

## Observations: Retrieval (1 of 2)

**Observations.** The similarity search for Question 2 returned four chunks. The best one starts with the “Three Leadership Characteristics” section: deep expertise, immersion in the details and willingness to collaboratively debate. It matched the meaning of the question, not just keywords.

**Why it matters.** The retriever found the passage holding the answer inside an 11-page report without reading the whole article. Semantic search works on a long business document.

**Business impact and action.** An analyst reaches the right page in seconds. Limit: the four results were two pairs of identical chunks, so only two distinct passages reached the model. Remove duplicates or use MMR before scaling.

## Observations: Embeddings (2 of 2)

**Embeddings.** The first two chunks became 1,536-dimension vectors (same length: True). Matching dimensions place every chunk in one vector space, so a question can be compared with each passage by similarity, even when the wording differs.

**Source traceability.** Each retrieved chunk keeps its source file, PDF page and file path. The response function adds a [Source PDF Page X] label before each passage, so the model can cite where its evidence came from.

**Business impact.** Analysts can check any claim against the original page before using it. Traceability is a design requirement, not an add-on.

# Data Preparation for RAG: Summary

## Dataset

HBR article “How Apple Is Organized for Innovation” (Podolny and Hansen, 2020), an 11-page PDF

## Splitting parameters

chunk\_size = 700 tokens  
chunk\_overlap = 100  
Token-based splitter, 18 chunks

## Embedding model

OpenAI text-embedding-ada-002  
1,536 dimensions, stored in a Chroma vector database

## RAG parameters

k = 4 retrieved chunks  
max\_tokens = 500, temperature = 0  
top\_p = 0.95, model gpt-4o-mini

# Question Answering using RAG: Purpose

**Purpose.** Answer the three project questions from the article itself. The retriever finds the four closest chunks, and gpt-4o-mini answers only from them and cites the PDF pages.

**Why this is important.** The baseline and prompting both answered from memory and could not be verified. RAG changes the evidence the model can use, so it is the first approach whose answers can be checked against the source.

**Business impact.** Analysts get answers with page references in seconds, can verify them before use and avoid reading the full report. This is the version that could be trusted in a research workflow.

# Question Answering using RAG: Prompts

## System prompt

- Answer only from the retrieved HBR context
- No outside knowledge or invented facts
- Say when the context is insufficient
- Follow the requested format (three bullets for Q2)
- Cite the supporting PDF page numbers

## User prompt

- **Retrieved context:** top 4 chunks, each labeled [Source PDF Page X]
- **Question:** the analyst's question
- **Instructions:** use only the context, follow the format, be concise, cite pages

**Retrieval:** similarity search,  $k = 4$     **Model:** gpt-4o-mini  
**Generation:** temperature 0, max\_tokens 500, top\_p 0.95

# RAG: Q1 Answer

**Question 1 (exact wording):** Who are the authors of this article and who published this article ?

```
response_with_rag_1 = generate_rag_response(question_1)
```

**Answer.** “The authors of this article are Joel M. Podolny and Morten T. Hansen. The article is published by Harvard Business Review. Supporting PDF page: 3.”

**For comparison.** Correct. The article’s byline names Podolny (Dean) and Hansen (Faculty) of Apple University, and Harvard Business Review. The baseline refused and prompting named the wrong authors.

# RAG: Q1 Output

Question 1: Who are the authors of this article and who published this article ?

```
In [54]: response_with_rag_1 = generate_rag_response(question_1)
response_with_rag_1
```

```
/tmp/ipykernel_22701/3338505941.py:11: LangChainDeprecationWarning: The method 'BaseRetriever.get_relevant_documents' was deprecated in langchain-core 0.1.46 and will be removed in 1.0. Use :meth:`~invoke' instead.
```

```
relevant_document_chunks = retriever.get_relevant_documents(
```

```
Out[54]: 'The authors of this article are Joel M. Podolny and Morten T. Hansen. The article is published by Harvard Business Review. \n\nSupporting PDF page: 3.'
```

**Notebook cell In [54].** The call, a LangChain deprecation warning (get\_relevant\_documents is being replaced by invoke; it does not change the result), and the raw output: the correct authors and “Supporting PDF page: 3.”

# RAG: Q1 Observations

**Observations.** RAG named Joel M. Podolny and Morten T. Hansen and Harvard Business Review, and cited PDF page 3, where the authors are printed. The answer is correct and matches the retrieved text. The pink box in the notebook is a deprecation warning, not an error.

**Why it matters.** This is the question the baseline refused and prompting got wrong. Retrieval turned a refusal and a false answer into a verified one.

**Business impact and action.** An analyst can cite the authors with confidence and check page 3 in seconds. Keep page citations in every answer.

## RAG: Q2 Answer

**Question 2 (exact wording):** List down the three leadership characteristics in bulleted points and explain each one of the characteristics under two lines.

```
response_with_rag_2 = generate_rag_response(question_2)
```

### **Answer (lightly shortened).**

**Deep expertise:** specialized knowledge in their fields, allowing them to engage meaningfully and make informed decisions.

**Immersion in details:** deeply involved in the specifics of their functions, understanding the intricacies of the work.

**Willingness to collaboratively debate:** open to debating ideas across functions, which enhances decision-making.

[Source PDF Page 6]

# RAG: Q2 Output

Question 2: List down the three leadership characteristics in bulleted points and explain each one of the characteristics under two lines.

```
In [55]: response_with_rag_2 = generate_rag_response(question_2) #Complete the code to pass the user input
response_with_rag_2
```

```
Out[55]: '- **Deep expertise**: Leaders at Apple are expected to have specialized knowledge in their respective fields, allowing them to engage meaningfully in their functions and make informed decisions.\n\n- **Immersion in details**: Managers are required to be deeply involved in the specifics of their functions, ensuring that they understand the intricacies of the work being done.\n\n- **Willingness to collaboratively debate**: Apple leaders must be open to discussing and debating ideas across different functions, fostering a collaborative environment that enhances decision-making.\n\n[Source PDF Page 6]'
```

**Notebook cell In [55].** The call and the raw output: three bullets, Deep expertise, Immersion in details and Willingness to collaboratively debate, each with a short explanation, ending with [Source PDF Page 6].

# RAG: Q2 Observations

**Observations.** All three characteristics match the article: deep expertise, immersion in the details and willingness to collaboratively debate. The format is as asked, three bullets with short explanations, and the answer ends with [Source PDF Page 6], the page that holds the section.

**Why it matters.** The baseline gave Vision, Empathy, Decisiveness and prompting gave generic traits. Only RAG matches the article, and it kept the prompt's format rules.

**Business impact and action.** The analyst can report Apple's leadership model correctly and verify it on page 6. Use prompt rules for format and retrieval for content.

# RAG: Q3 Answer

**Question 3 (exact wording):** Can you explain specific examples from the article where Apple's approach to leadership has led to successful innovations?

```
response_with_rag_3 = generate_rag_response(question_3)
```

**Answer (shortened; full text on the output slide).**

**1. Expert Leadership:** Roger Rosner, head of Apple's software applications business, exemplifies "experts leading experts"; his teams create Pages, Numbers and Keynote. (Page 7)

**2. Functional Organization:** 600-plus camera hardware experts led by Graham Townsend work together, not scattered across product lines. (Page 7)

**3. Discretionary Leadership Model:** leaders focus on the areas that create the most value. (Page 10)

# RAG: Q3 Output

Question 3: Can you explain specific examples from the article where Apple's approach to leadership has led to successful innovations?

```
In [56]: response_with_rag_3 = generate_rag_response(question_3) #Complete the code to pass the user input
response_with_rag_3
```

```
Out[56]: 'Apple\'s approach to leadership has led to successful innovations in several ways:\n\n1. Expert Leadership: Roger Rosner, who heads Apple\'s software application busi-
ness, exemplifies the principle of "experts leading experts." His deep expertise in software engineering has allowed him to effectively lead teams that create innovativ
e applications like Pages, Numbers, and Keynote. This leadership approach fosters an environment where specialists can collaborate and refine their innovations, resultin
g in high-quality products. (Source PDF Page 7)\n\n2. Functional Organization: Apple\'s structure allows for specialists, such as the more than 600 camera hardware ex-
perts led by Graham Townsend, to work together rather than being scattered across product lines. This concentration of expertise enhances problem-solving capabilities an
d drives innovation in camera technology across various devices, such as iPhones and iPads. (Source PDF Page 7)\n\n3. Discretionary Leadership Model: As Apple has gr-
own, the discretionary leadership model has been implemented to help leaders focus on areas that create the most value. This model encourages leaders to immerse themselv
es in critical details while delegating less critical tasks, which has enabled Apple to maintain its innovative edge across all areas of the company, not just product de-
velopment. (Source PDF Page 10)\n\nThese examples illustrate how Apple\'s leadership approach, emphasizing expertise and collaboration, has been integral to its innovati-
on success.'
```

**Notebook cell In [56].** The call and the raw output: three numbered examples, each followed by “(Source PDF Page 7)” or “(Source PDF Page 10)”, then a closing sentence on expertise and collaboration.

# RAG: Q3 Observations

**Observations.** Every example comes from the article: Roger Rosner's software team, the 600-plus camera experts led by Graham Townsend, and the discretionary leadership model. Each is tied to leadership and cites a page. Not covered: the iPhone 7 Plus dual-lens decision on page 6.

**Why it matters.** The baseline and prompting listed well-known products from memory. RAG gave the article's own evidence, and the page labels let a reader confirm each point.

**Business impact and action.** Analysts can cite checkable examples. To widen coverage, test a higher k or MMR.

# RAG: Evaluation

| Criterion           | Q1 Authors      | Q2 Traits           | Q3 Examples          |
|---------------------|-----------------|---------------------|----------------------|
| <b>Accuracy</b>     | Correct authors | Correct traits      | From the article     |
| <b>Relevance</b>    | Direct          | Fully relevant      | Fully relevant       |
| <b>Completeness</b> | Complete        | Complete            | Good, not exhaustive |
| <b>Instructions</b> | Followed        | Followed: 3 bullets | Followed: linked     |
| <b>Traceability</b> | Page 3          | Page 6              | Pages 7 and 10       |
| <b>vs prompting</b> | Wrong → correct | Generic → article   | Memory → source      |

**Business impact:** RAG was the only approach with correct, traceable answers to all three questions. The remaining risk is retrieval quality, such as duplicate chunks.

# Observations: RAG (1 of 4)

**What was done.** A document-retrieval layer was added between the user's question and the language model. Unlike the baseline and prompt-engineered approaches, the model no longer had to answer only from pretrained knowledge.

## The workflow

1. Loaded the 11-page HBR PDF with PyMuPDFLoader
2. Split it into 18 chunks with a token-based splitter
3. Used 700-token chunks with 100 tokens of overlap
4. Embedded each chunk as a 1,536-dimension vector
5. Stored the vectors and metadata in Chroma
6. Retrieved the four closest chunks per question ( $k=4$ )
7. Passed the chunks and PDF pages to gpt-4o-mini
8. Told the model to answer only from them and cite pages

## Observations: RAG (2 of 4)

**Comparison with the baseline LLM.** The baseline sent the questions to gpt-4o-mini without the HBR article. Answers depended on pretrained knowledge, there was no guarantee they came from the article, source verification was not possible and hallucination risk was higher.

**What RAG changes.** RAG retrieves the relevant HBR content before the model answers. The key improvement is not a more polished answer; it changes the evidence available to the model.

**Business impact.** The baseline shows what the model already knows; only RAG gives a reliable document-analysis workflow.

## Observations: RAG (3 of 4)

**Comparison with prompt engineering.** Prompt engineering improved the behavior of the same LLM through detailed instructions on accuracy, structure, conciseness, formatting, uncertainty and avoiding unsupported claims. Lowering temperature from 0.7 to 0.2 made answers more focused.

**The limit.** Prompt engineering still did not give the model the HBR document. It improved how the model answered, but RAG improved what evidence the model could use.

**Business impact.** RAG keeps the benefits of a well-designed prompt and adds retrieved source material.

## Observations: RAG (4 of 4)

**Overall assessment.** RAG is the strongest of the three approaches because it combines semantic retrieval, prompt engineering, source-document context, deterministic generation (temperature 0), metadata preservation and natural-language answers.

**Important limitations.** RAG improves grounding but does not remove all risk. Quality depends on retrieval: if the right passage is not retrieved, the model cannot use it. The similarity search also returned duplicate chunks.

**Action.** Deduplicate, test k, use MMR, add metadata filters and rerank.

# RAG: Summary

| Question                        | Answer                                                                                                                   | Observation                                                       |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Q1 Authors and publisher</b> | Joel M. Podolny and Morten T. Hansen; published by Harvard Business Review (p. 3)                                        | Correct and cited; the prompt-only answer named the wrong authors |
| <b>Q2 Leadership traits</b>     | Deep expertise; immersion in the details; willingness to collaboratively debate (p. 6)                                   | Matches the article's three characteristics, in three bullets     |
| <b>Q3 Innovation examples</b>   | Roger Rosner's software apps team; Graham Townsend's 600+ camera experts; the discretionary leadership model (pp. 7, 10) | Specific examples from the article, each with page citations      |

**RAG answered all three questions from the article, with page references.**

# Comparison of the Three Approaches (1 of 2)

| Capability                      | Baseline | Prompting | RAG                                       |
|---------------------------------|----------|-----------|-------------------------------------------|
| Uses pretrained model knowledge | Yes      | Yes       | Yes, but constrained by retrieved context |
| Uses the assigned HBR document  | No       | No        | Yes                                       |
| Structured instructions         | Minimal  | Strong    | Strong                                    |
| Semantic retrieval              | No       | No        | Yes                                       |
| Vector database                 | No       | No        | Yes                                       |

**Why it matters:** RAG is the only approach that uses the document, retrieves by meaning and stores it for reuse.

## Comparison of the Three Approaches (2 of 2)

| Capability                                | Baseline | Prompting        | RAG                                      |
|-------------------------------------------|----------|------------------|------------------------------------------|
| Source-page traceability                  | No       | No               | Yes                                      |
| Document-specific grounding               | Low      | Low              | High                                     |
| Hallucination risk                        | Higher   | Reduced somewhat | Lower when relevant context is retrieved |
| Ability to verify claims                  | Limited  | Limited          | Stronger                                 |
| Suitable for enterprise document analysis | Limited  | Moderate         | Strong                                   |

**Business impact:** RAG is strongest for enterprise document analysis when retrieval finds the right passage.

# Three Approaches: How Each One Works

| Approach                  | How it works                                         | Settings             | Main limit                   |
|---------------------------|------------------------------------------------------|----------------------|------------------------------|
| <b>Baseline LLM</b>       | Question sent straight to gpt-4o-mini                | Temperature 0.7      | No article, no sources       |
| <b>Prompt engineering</b> | Detailed system prompt, same model                   | Temperature 0.2      | Still no article             |
| <b>RAG</b>                | Four closest chunks from the PDF added to the prompt | k = 4, temperature 0 | Depends on retrieval quality |

**Why it matters:** only RAG changes the evidence the model sees. The other two change only how it answers.

# Three Approaches: Results by Question

| Question           | Baseline                             | Prompting                                          | RAG                                                                |
|--------------------|--------------------------------------|----------------------------------------------------|--------------------------------------------------------------------|
| <b>Q1 Authors</b>  | Refused to answer                    | Named the wrong authors                            | Podolny and Hansen, HBR; page 3                                    |
| <b>Q2 Traits</b>   | Vision, empathy, decisiveness        | Visionary thinking, collaboration, decisive action | Deep expertise, immersion in details, collaborative debate; page 6 |
| <b>Q3 Examples</b> | Five well-known products from memory | iPhone, Apple Watch, iPad; no source               | Rosner, Townsend, discretionary model; pages 7 and 10              |

**Business impact:** only the RAG column can be checked against the article.

# Overall Observations

**Baseline.** Fluent but unverifiable. It refused Q1, listed generic traits for Q2 and used well-known products from memory for Q3.

**Prompting.** Better format and tone, but the same missing evidence. It named the wrong authors for Q1 and gave generic traits for Q2.

**RAG.** The only approach that matched the article on all three questions. Every answer carried a PDF page number (3, 6, 7 and 10).

# Business Impact

**Speed.** An analyst gets a sourced answer from an 11-page report in seconds instead of reading it end to end. The same pipeline can index larger collections.

**Risk.** Page citations let a reader check a claim before it enters an investment or strategy decision. The baseline and prompting gave no way to do that.

**Trust.** Wrong authors in the prompting answer show a confident reply can still be false. Grounding answers in the source makes an assistant safe to use for research.

# Why This Project Matters

**The problem.** Analysts face long, dense reports. Manual review is slow and can miss details that matter to a decision.

**The proof.** A working prototype on a real HBR article shows that plain-language questions can return correct, cited answers without reading the whole report.

**The lesson.** Evidence matters more than fluency. A model that sounds right is not the same as one that is right, and only a traceable source shows the difference.

# Overall Business Insights

**Retrieval decides quality.** The model can only use what the retriever returns. Chunk size, overlap, k and duplicate chunks all shape the final answer.

**Prompting and RAG work together.** Prompt rules controlled format and tone, and retrieval supplied the facts. A production tool needs both.

**Experts keep the decision.** The article's own lesson applies: Apple places decisions with experts. Analysts should use RAG to find evidence and keep the final judgment.

# Applying This to Other Industries

| Industry                     | Documents                                | Benefit                                 |
|------------------------------|------------------------------------------|-----------------------------------------|
| <b>Finance and investing</b> | Earnings reports, filings, due diligence | Faster review with page-level citations |
| <b>Legal</b>                 | Contracts, policies and case files       | Clauses traced to the source page       |
| <b>Healthcare</b>            | Clinical guidelines and protocols        | Answers limited to approved documents   |
| <b>Manufacturing</b>         | Safety, quality and maintenance manuals  | Consistent procedure lookup for staff   |
| <b>Customer support</b>      | Product and policy documents             | Consistent answers agents can verify    |

# Overall Recommendations

1. **Use RAG with prompt rules** for any question about a specific document.
2. **Keep PDF page citations** in every answer so claims can be checked.
3. **Have a person verify** conclusions that feed investment or strategy decisions.
4. **Measure retrieval separately** (precision, recall) from answer quality.
5. **Tune before scaling:** chunk size, k, duplicate handling, MMR and reranking.
6. **Extend to many documents** with filters by date, company and source.
7. **Test agentic RAG** against the same three questions and criteria before adopting it.

**Key point:** RAG supports analyst judgment; it does not replace it.

# Possible Applications and Future Study

| Area                      | What to study                                                          | Why                                   |
|---------------------------|------------------------------------------------------------------------|---------------------------------------|
| <b>Multiple documents</b> | Questions across a collection of reports, with filters                 | Matches how analysts work             |
| <b>Retrieval tuning</b>   | Chunk size, overlap, k, MMR and reranking                              | Fewer duplicate chunks                |
| <b>Evaluation</b>         | Retrieval precision and recall, faithfulness, citation accuracy        | Measures quality, not fluency         |
| <b>Analyst workflow</b>   | Conversation history and side-by-side sources                          | Supports follow-up questions          |
| <b>Agentic RAG</b>        | Rewrite the question, retrieve in several steps and check the evidence | Finds passages a single search misses |

**Application:** a research assistant for a team that reviews many reports.

# Single-Pass RAG vs Agentic RAG

| Aspect                         | Single-pass RAG (tested)                             | Agentic RAG (to test)                      |
|--------------------------------|------------------------------------------------------|--------------------------------------------|
| <b>Retrieval</b>               | One search, $k = 4$                                  | Searches, checks the result and retries    |
| <b>The question</b>            | Used as written                                      | Can be rewritten or split into parts       |
| <b>Multi-passage questions</b> | Can miss a passage, such as the page 6 example in Q3 | Can combine evidence from several searches |
| <b>Cost and speed</b>          | One model call, fast                                 | More calls, slower and costlier            |
| <b>Traceability</b>            | Page label on every chunk                            | Page citation must be kept at every step   |

**Next step:** run Agentic RAG on the same three questions and compare.

# Metadata and Data Freshness

## Metadata: better answers

- Filter before searching, by document, date or section
- Richer citations: title, author, date and page
- Fill gaps: this PDF's title and author fields are empty
- Test filtering against reranking on precision and recall

## Fresh data: current answers

- Answers are only as current as the index
- Store publication date and version as metadata
- Replace superseded files, do not only add
- Re-index on a schedule, with a named owner

**Example:** this article's figures end in 2019, so RAG would repeat them as current.

# Conclusion

**What we did.** Asked three questions about the HBR article three ways: a plain LLM, a prompt-engineered LLM and a RAG pipeline (18 chunks, text-embedding-ada-002, Chroma, gpt-4o-mini).

**What we found.** Only RAG answered all three correctly and with page references: Podolny and Hansen as authors, the three traits on page 6, and examples from pages 7 and 10.

**What it means.** RAG is a practical way to cut document-review time while keeping evidence visible. It supports analyst judgment and does not replace it.

# APPENDIX

# Data Background and Contents

- **Source:** Harvard Business Review, November–December 2020, “How Apple Is Organized for Innovation”
- **Authors:** Joel M. Podolny (Dean) and Morten T. Hansen (Faculty), both of Apple University
- **Format:** 11-page PDF reprint (R2006F), dated 2020; its figures end in 2019
- **Topics:** Apple's functional organization, why expertise aligns with decision rights, three leadership characteristics, the discretionary leadership model
- **Scale:** Apple grew from about 8,000 employees and \$7 billion revenue in 1997 to 137,000 employees and \$260 billion in 2019

# Project Instructions and Steps

## Project instructions

- Read the problem statement before starting
- Run the commented notebook in order
- Fill each blank with code that matches its comment
- Add results and observations to the presentation
- Put graded computational details in the Appendix

## Steps followed

1. Install and import the libraries
2. Connect to the OpenAI API
3. Ask the three questions with the baseline LLM
4. Repeat with a system prompt
5. Load, chunk, embed and store the PDF
6. Retrieve chunks and answer with RAG
7. Compare the three approaches
8. Review insights, recommendations and next steps

# Code: Setup and Baseline LLM

```
# Versions: langchain 0.3.27, chromadb 1.0.15, pymupdf 1.26.3, tiktoken 0.9.0
import os, json
from openai import OpenAI

# config.json holds OPENAI_API_KEY and OPENAI_API_BASE
with open("config.json", "r") as file:
    config = json.load(file)
os.environ["OPENAI_API_KEY"] = config.get("OPENAI_API_KEY")
os.environ["OPENAI_BASE_URL"] = config.get("OPENAI_API_BASE")
client = OpenAI()

def response(user_prompt, max_tokens=500, temperature=0.7, top_p=1.0):
    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": user_prompt}],
        max_tokens=max_tokens,
        temperature=temperature,
        top_p=top_p
    )
    return completion.choices[0].message.content
```

# Code: Questions and Calls

```
question_1 = "Who are the authors of this article and who published this article ?"  
question_2 = ("List down the three leadership characteristics in bulleted points and "  
             "explain each one of the characteristics under two lines.")  
question_3 = ("Can you explain specific examples from the article where Apple's "  
             "approach to leadership has led to successful innovations?")  
  
# 1. Baseline LLM (question only)  
base_prompt_response_1 = response(question_1)  
base_prompt_response_2 = response(question_2)  
base_prompt_response_3 = response(question_3)  
  
# 2. LLM with prompt engineering (system prompt + question)  
response_with_prompt_eng_1 = response(system_prompt, question_1)  
response_with_prompt_eng_2 = response(system_prompt, question_2)  
response_with_prompt_eng_3 = response(system_prompt, question_3)  
  
# 3. RAG (retrieved context + system prompt + question)  
response_with_rag_1 = generate_rag_response(question_1)  
response_with_rag_2 = generate_rag_response(question_2)  
response_with_rag_3 = generate_rag_response(question_3)
```

# Code: Prompt Engineering

```
system_prompt = """
You are a careful business research assistant answering questions about the
Harvard Business Review article "How Apple Is Organized for Innovation."
Provide accurate, concise, and well-structured responses.
Follow these instructions:
1. Answer the user's question directly.
2. Do not invent facts or unsupported details.
3. If you are uncertain about a factual detail, clearly state the uncertainty.
4. Follow the requested response format exactly.
5. For the leadership characteristics question, provide exactly three bullet points.
6. Keep each leadership-characteristic explanation concise and within two lines.
7. For questions about innovation, identify specific examples and explain how
Apple's leadership approach contributed to the innovation.
8. Prioritize factual accuracy, relevance, clarity, and completeness over creativity.
"""

def response(system_prompt, user_prompt, max_tokens=500, temperature=0.2, top_p=1.0):
    # same chat.completions.create call as the baseline, with a system message:
    # messages=[{"role": "system", "content": system_prompt},
    #           {"role": "user", "content": user_prompt}]
```

# Code: Data Preparation

```
from langchain_community.document_loaders import PyMuPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

pdf_path = ("/content/drive/MyDrive/Apple HBR Report Project/"
            "HBR_How_Apple_Is_Organized_For_Innovation.pdf")
pdf_loader = PyMuPDFLoader(pdf_path)

text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cll100k_base", chunk_size=700, chunk_overlap=100)
document_chunks = pdf_loader.load_and_split(text_splitter)    # 18 chunks

embedding_model = OpenAIEmbeddings(
    openai_api_key=OPENAI_API_KEY, openai_api_base=OPENAI_API_BASE)
embedding_1 = embedding_model.embed_query(document_chunks[0].page_content)
print(len(embedding_1))    # 1536

out_dir = "apple_hbr_chroma_db"
vectorstore = Chroma.from_documents(
    document_chunks, embedding_model, persist_directory=out_dir)
```

# Code: RAG System Prompt

```
qna_system_message = """
You are a retrieval-augmented question-answering assistant for business analysts.
Answer each question using only the retrieved context from the Harvard Business Review
article "How Apple Is Organized for Innovation."
Follow these rules:
1. Use only information contained in the retrieved context.
2. Do not add facts from outside knowledge.
3. Do not invent unsupported information.
4. If the retrieved context does not contain enough information to answer the question,
clearly state that the available context is insufficient.
5. Answer the question directly, accurately, and concisely.
6. Follow the requested response format exactly.
7. For the leadership characteristics question, provide exactly three bullet points.
8. For innovation examples, explain how Apple's leadership approach contributed to
the resulting innovation.
9. When source-page information is available, identify the page or pages supporting
the answer.
"""
```

# Code: RAG User Prompt and Retriever

```
retriever = vectorstore.as_retriever(  
    search_type="similarity",  
    search_kwargs={"k": 4}  
)  
  
qna_user_message_template = """  
Use the retrieved context below to answer the question.  
Retrieved Context:  
{context}  
Question:  
{question}  
Instructions:  
- Base your answer only on the retrieved context.  
- Do not add unsupported information.  
- Follow the formatting requirements in the question.  
- Be concise but complete.  
- Identify the supporting PDF page number(s) when page information is available.  
"""  
  
# Retrieval check: the four closest chunks to question 2  
vectorstore.similarity_search(question_2, k=4)
```

# Code: RAG Response Function

```
def generate_rag_response(user_input, k=4, max_tokens=500, temperature=0, top_p=0.95):
    global qna_system_message, qna_user_message_template
    relevant_document_chunks = retriever.get_relevant_documents(query=user_input)
    context_list = []
    for d in relevant_document_chunks:
        page = d.metadata.get("page")
        page_number = page + 1 if isinstance(page, int) else "Unknown"
        context_list.append(f"[Source PDF Page {page_number}]\n{d.page_content}")
    context_for_query = "\n\n".join(context_list)
    user_message = qna_user_message_template.replace('{context}', context_for_query)
    user_message = user_message.replace('{question}', user_input)
    try:
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "system", "content": qna_system_message},
                      {"role": "user", "content": user_message}],
            max_tokens=max_tokens, temperature=temperature, top_p=top_p)
        response = response.choices[0].message.content.strip()
    except Exception as e:
        response = f'Sorry, I encountered the following error: \n {e}'
    return response
```



**Power Ahead!**

